Emploid AI Assistant & Resume Parser

1. Emploid Assistant Copilot

Emploid includes an inline job-search assistant that functions as a copilot for the user. It can recommend relevant jobs, answer questions about application status, suggest follow-up activities, and explain Listing Trust Scores.

Assistant API Endpoint (`/api/assistant/route.ts`)

The assistant exposes a POST endpoint which streams responses back to the client using **Server-Sent Events (SSE)**.

- **Payload Verification:** It validates the incoming request parameters using a Zod schema `AssistantRequest`:
- `messages`: An array of chat history objects with `role` ('user' or 'assistant') and `content` (up to 3,000 characters), capped at 12 messages to keep context sizes reasonable.
- `resumeProfile`: Optional parsed profile of the user's resume.
- `currentPage`: The user's current surface (e.g. 'home', 'jobs', 'tracker', 'about', 'blog').
- `trackerApplications`: Up to 30 of the user's active tracked jobs.
- `jobs`: Up to 80 jobs currently loaded in the client list.
- `filters`: The active filtering criteria.
- **Intent Detection (`shouldSearchJobs`):** The API dynamically scans the latest user prompt and filter query to determine if a job search is requested. It triggers a search if terms like "find", "search", "recommend", "best fit", or specific job titles are detected.
- **Pre-fetch Google Jobs integration:** If the user request implies a search, the backend calls the Google Jobs API (`/api/google-jobs`) to pull up to 8 live relevant openings *before* invoking the LLM. This provides fresh, real-time results for the LLM.
- **LLM Context Construction:** The API builds a highly targeted system instruction prompt and injects the user's context (e.g., active resume summary, active filters, available jobs, and tracker applications).
- **Client Streaming response:** The response is returned as a UTF-8 `text/event-stream` stream, sending incremental chat delta chunks and terminating with a JSON object containing the recommended jobs array, search query parameters, and completion status.

LLM Prompt Constraints

The assistant runs on a custom model setup (`ASSISTANT_MODEL`, falling back to `llama-3.1-8b-instant` on Groq). The system instructions dictate strict formatting constraints:

- **No Hallucinated Jobs:** The assistant can only recommend jobs present in the `availableJobs` context parameter. It must never invent job listings or database IDs.
- **Clickable Links:** Recommended jobs must be referenced with their exact title, company, location,

and trust score, containing markdown links formatted like `Job Title`.

- **Application Tracker context:** The assistant references tracked applications only when queries touch on stages, follow-up timelines, or status tracking.
- **Conciseness:** Prompts demand highly actionable, direct, and warm responses.

---

2. Resume Parsing Engine

To enable personalized semantic job matching, Emploid allows users to upload their resumes. The resume is parsed on the server into structured data.

Parsing Endpoint (`/api/resume/parse/route.ts`)

The parsing endpoint handles PDF/text uploads:

- **Zod Input Schema (`ResumeParseRequest`):** Expects a JSON body containing `text` (extracted text of the resume, between 40 and 24,000 characters) and the optional `fileName`.
- **Structured JSON Mode:** The parser requests JSON format from the LLM using the following schema constraints:

```
{
  "summary": "Short paragraph summarizing profile",
  "focusRoles": ["Array of inferred target job roles"],
  "skills": ["Array of technical/transferable skills"],
  "seniorityLevel": "junior | mid | senior",
  "yearsExperience": 5.0,
  "preferredWorkModes": ["remote", "hybrid", "onsite"],
  "preferredLocations": ["Chicago, IL", "New York, NY"],
  "workModes": ["Alias list for compatibility"],
  "locations": ["Alias list for compatibility"],
  "chips": ["Key summary tags"]
}
```

- **LLM Parsing Constraints:** The parser runs at a low temperature (`0.15`) to reduce hallucinations. System prompts restrict the model to *only* inferring transferable skills and roles from the provided text, explicitly banning the creation of fake credentials, employers, or degrees.
- **Normalization Helper (`normalizeProfile`):** Sanitizes and clamps the LLM output:
- Limits `focusRoles` to 4 items and `skills` to 10.
- Generates up to 6 custom `chips` tags representing the core highlights.
- Resolves work modes and locations for backwards client compatibility.
- Syncs the parsed profile with the user's database records to enable semantic job suggestion overlays.